RewardClaimV1 — Audit Report

Findings status and accepted-risk rationale

Date	November 6, 2026
Contract	`contracts/rewards/RewardClaimV1.sol`
Revision	branch `claim-reward`, commit `83ea4e7`. Findings fixed before this revision are not included.
Environment	solc 0.8.35 (viaIR, evm prague), OpenZeppelin Contracts 5.6.1, Ethereum mainnet
Purpose	Payout of loyalty prizes in LIT (ERC20) against a backend EIP-712 signature. One campaign, 1–2 weeks, a pool of a few thousand dollars, prizes of a few dollars each.
Method	AI-assisted audit (Claude) in two independent passes with no shared conclusions; cross-checked against the OZ 5.6.1 sources (`ECDSA`, `SignatureChecker`, `EIP712`, `SafeERC20`) and `HermesDelegateV1`; 7 PoC scenarios on hardhat; test suite 13/13. Decisions on each finding: Andrei Chupin.

Summary

0	1	2	3
Critical / High	Medium — accepted	Low — accepted	Informational — accepted

The contract itself cannot lose funds, pay a prize twice or pay someone else's prize. Every finding in this report is operational or integration-level: the contract cannot enforce backend invariants, and adding privileged roles (owner, pause, cancel) for them would add code and keys that would need protecting in turn. Given the scale of the campaign, all of them are accepted; each one records why it is not fixed and the compensating measures.

ID	Severity	Finding	Status
M-1	Medium	Double payout if the backend issues a new `claimId` per signing request	Accepted · backend
L-1	Low	`spent` is keyed by `claimId` alone	Accepted · backend
L-2	Low	A leaked signer key exposes the whole pool, no pause	Accepted · risk
I-1	Info	Signature format: 65-byte `r‖s‖v` only	Accepted · ethers format
I-2	Info	No revocation of an issued proof	Accepted · risk
I-3	Info	Token assumptions	Accepted · LIT is standard

Findings

M-1 Medium Double payout if the backend issues a new `claimId` per signing request — Accepted

WHERE

`claim()`: the only replay protection is `spent[claimId]`.

SCENARIO

A user opens the Claim screen twice or retries after a timeout. If the backend generates a random `claimId` per request, it hands out two valid proofs for one prize and `claim()` pays both. The contract has no notion of a "prize" or of per-account limits.

> Why not fixed
> The invariant "one prize — one `claimId`" lives in the backend; moving it into the contract would mean moving the prize database there too. It is a backend bug that does not hurt the user: in the worst case we overpay, within the pool.

COMPENSATING MEASURES

`claimId` is derived deterministically from the prize's primary key, e.g. `base32_lower_nopad(blake2b_160(prize_id))`; a repeated request yields the same `claimId` and changes only the `deadline`. Backend test: signing one prize twice yields the same `claimId`.

L-1 Low `spent` is keyed by `claimId` alone — Accepted

WHERE

`mapping(string claimId => bool) public spent`.

SCENARIO

The backend mistakenly signs one `claimId` for two users. The first one gets paid, the second one gets `ClaimSpent` until the backend issues another id. Only the backend can trigger this; a user cannot.

> Why not fixed
> The proposed `(account, claimId)` key widens the attack surface: the same `claimId` becomes valid for different accounts, and the same backend mistake turns from one blocked payout into several payouts. The current scheme fails closed instead of overpaying, which is safer for us. `claimId` uniqueness is a backend invariant, as in M-1.

COMPENSATING MEASURES

Global `claimId` uniqueness by construction (a hash of the prize's primary key), see M-1.

L-2 Low A leaked signer key exposes the whole pool, no pause

Accepted

WHERE

`signer` is immutable; pause, cancel and key rotation are absent by design (documented in the contract's NatSpec).

SCENARIO

The backend key leaks → the attacker signs a `Claim` to their own address for the whole balance and takes the pool. Nothing on-chain can stop it; the only option is to take the remainder first with the same self-claim.

> Why not fixed
>
> The campaign runs 1–2 weeks with a pool of a few thousand dollars; the damage is bounded by the campaign size and accepted. A pause or key rotation would add a second privileged role with the same key-custody risks.

COMPENSATING MEASURES

A dedicated key that is never imported anywhere or delegated; funding in tranches where the process allows; monitoring of `Claimed` events: an `amount` above the maximum prize, or an `account` missing from the backend database → alert and an immediate self-claim of the remainder.

I-1 Info Signature format: 65-byte `r‖s‖v` only

Accepted

WHAT

`ECDSA.tryRecoverCalldata` (OZ 5.6.1) accepts only 65 bytes with `v ∈ {27, 28}` and low-`s`. A 64-byte compact signature (EIP-2098) and `v ∈ {0, 1}` yield `InvalidSignature`.

> Why not fixed
>
> The backend signs with ethers `signTypedData`, which produces exactly this format. The behaviour is pinned by the test "rejects a 64-byte compact signature".

FOR THE FUTURE

When moving to KMS/HSM: normalise `s` to its low form, compute `v`, assemble 65 bytes.

I-2 Info No revocation of an issued proof

Accepted

WHAT

A proof is retired only by `spent` or `deadline`. If the same `claimId` is re-signed with a smaller amount, the earlier proof with the larger amount stays valid until its `deadline`. A zero `amount` is accepted and spends the id.

> Why not fixed
>
> On-chain revocation needs an owner role or per-account nonces. The worst case is an overpayment on our side, within a pool of a couple of thousand dollars; the user is not affected.

COMPENSATING MEASURES

A short proof TTL (300 s); never lower a prize amount while an older proof is live; never issue zero-amount signatures.

I-3 Info Token assumptions

Accepted

WHAT

The contract assumes a plain ERC20: a fee-on-transfer token would make `Claimed.amount` overstate the payout; a pausable or blocklisting token could freeze the pool with no rescue path. Hooks (ERC-777/1363) are safe: `spent` is set before the transfer.

> Why not fixed
> LIT is a standard ERC20 with none of these traits.

Verified correct

- **EIP-712.** The `CLAIM_TYPEHASH` string matches the struct `Claim(address account,uint256 amount,string claimId,uint256 deadline)`; `string` is encoded as `keccak256(bytes(claimId))`; domain `("RewardClaim", "1", chainId, this)`; the digest matches ethers `TypedDataEncoder` (test).
- **Replay.** Binding to `msg.sender` — across users; `chainId` in the domain, rebuilt on a fork — across chains; `verifyingContract` — across deployments (test); `spent` — re-signed proofs. Signature malleability is irrelevant: replay is keyed by id, not by signature.
- **Reentrancy.** `spent` and the event are set before `safeTransfer`; no ETH paths (`claim` is not payable, no `receive` / `fallback`); re-entering with the same id → `ClaimSpent`.
- **`msg.sender` under EIP-7702.** `HermesDelegateV1._call` is a plain `CALL` from `address(this)`, so the inner `msg.sender == EOA` (PoC). A relayer or any third party submitting the bare proof gets `InvalidSignature`.
- **`SafeERC20.safeTransfer`.** Tokens with no return value and tokens returning `false` are handled; a failed transfer leaves `claimId` unspent (test).
- **Other.** `deadline` is inclusive, tested at the boundary; the constructor rejects a zero `token_` and a zero `signer_`; pragma pinned to 0.8.35; the contract's only state is `spent`.